Built with Go

Self-Healing Microservice Mesh

A Go-Based Control Plane with Reconciliation-Driven Fault Recovery

Tripti (2301010278) | Rizpah (2301010219) | Raghuveer (2301010268)
Ayush (2301010253) | Aman (2301010277) | Sarthak (2301010216)

First Evaluation

1 Problem Statement

Apps are split into many small services (microservices)

Any service can crash, hang or slow down

Fixing it by hand means downtime

Goal: let the system detect and fix failures by itself

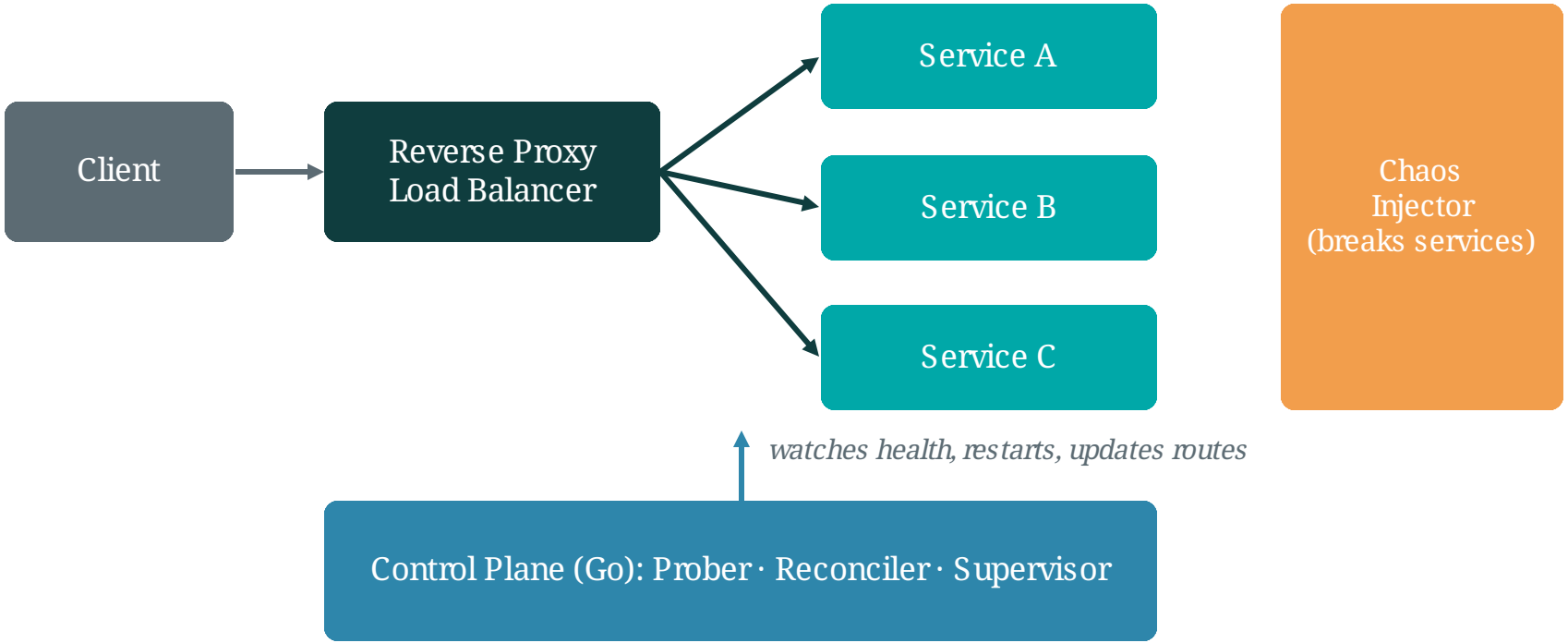
2 Our Solution

- Build deliberately fragile microservices
- Build a Go control plane that watches them
- It restarts failed services automatically
- It sends traffic only to healthy copies

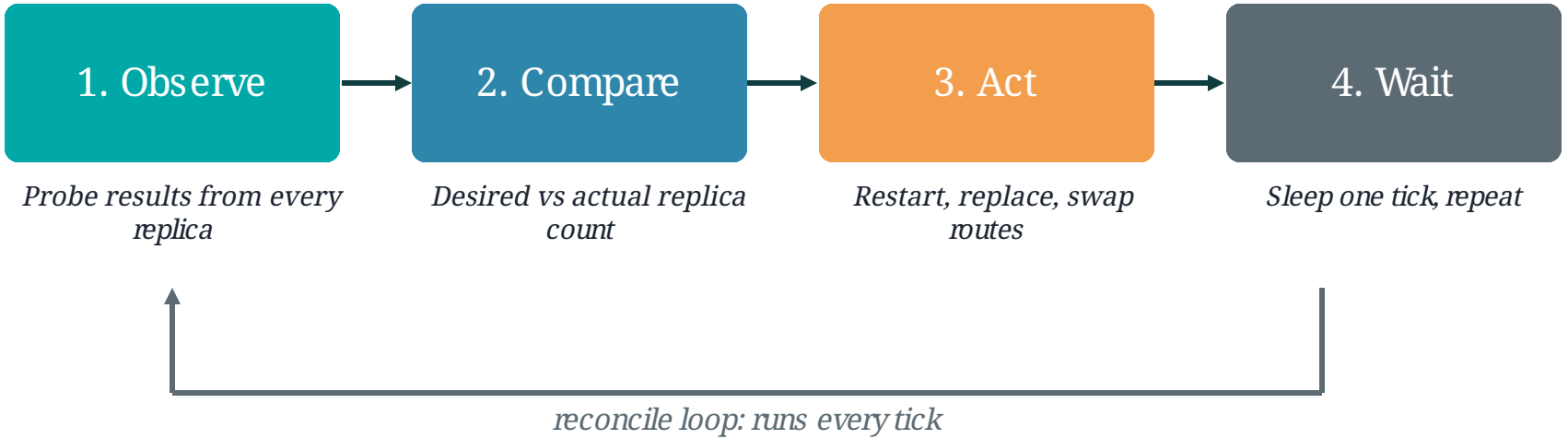
3 Why Go

- Goroutines: thousands of health checks at low cost
- Channels and context: safe communication and timeouts
- Standard library covers HTTP and process control
- Kubernetes and Docker are written in Go

4 System Architecture

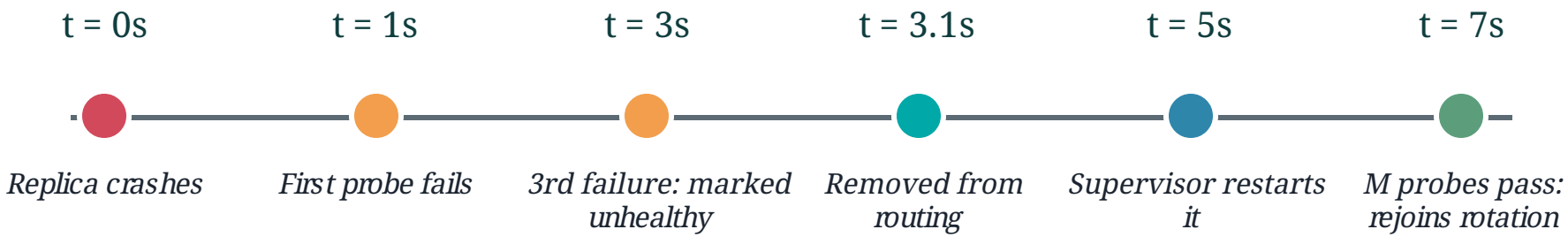


5 The Reconcile Loop (Flow)



Level-triggered: acts on current state, not on individual events

6 Anatomy of a Failure (Timeline)



Target: users see no errors once step 4 completes (about 3 seconds)

7 Go Concepts We Use

Goroutines

One lightweight worker per service health check

Channels

Probes send results to the controller safely

Context

Timeouts so a hung service cannot block us

Mutex / atomic

Safe updates to the routing table

8 Plan and Progress

Phase 1	Phase 2	Phase 3	Phase 4	Phase 5	Phase 6
Fragile services + chaos	*Health prober*	*Supervisor (restart)*	*Proxy + rerouting*	*Metrics dashboard*	*Testing + report*

Phases 1-2: the base | Phases 3-4: self-healing | Phases 5-6: prove it works

9 Demo, Outcomes and Future Scope

Demo: kill a service live while traffic is running

Result: detected, rerouted and restarted automatically

We measure detection time, recovery time, success rate

Future: leader election, auto-scaling, dashboard